

APAI LAB 03: DNN Shrinking and Quantization

Călin Diaconu – calin.diaconu2@unibo.it
Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it
Ihor Sachok – ihor.sachok@unibo.it
Marco La Barbera - marco.labarbera2@unibo.it



01. Lab Information

01. Lab Information – Objective of the Class

- **Intro**: DNN shrinking techniques (fewer channels, fewer layers, larger stride) and quantization (PTQ, QAT, true quantization)

- **Tasks**: make the CNN from Lab 02 smaller and integer-only:
  - Halve the number of channels and retrain the shrunk network
  - Quantize it: first a tensor by hand, then the whole network with Brevitas (PTQ and QAT)
  - Convert it to a true integer network with DeepQuant and inspect it with Netron
  - Complete the Tasks in the Python Notebook

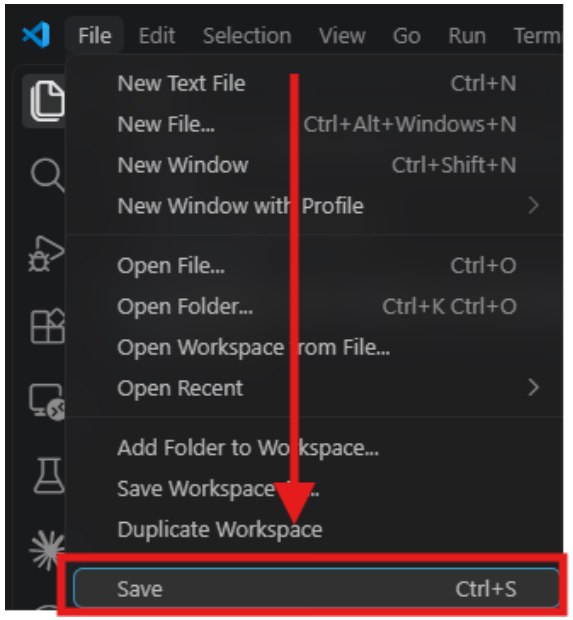
- **Programming language**: Python
- **Lab duration**: 3h
- **AI policy**: we will not prevent you from using AI in solving the assignments, but the lab is meant to be a hands-on learning experience. To keep it useful, we have included an .md file (also used in a Stanford class), that should be automatically read by AI agents, in order to prevent them to give you a direct solution and rather guide you towards finding it yourself.

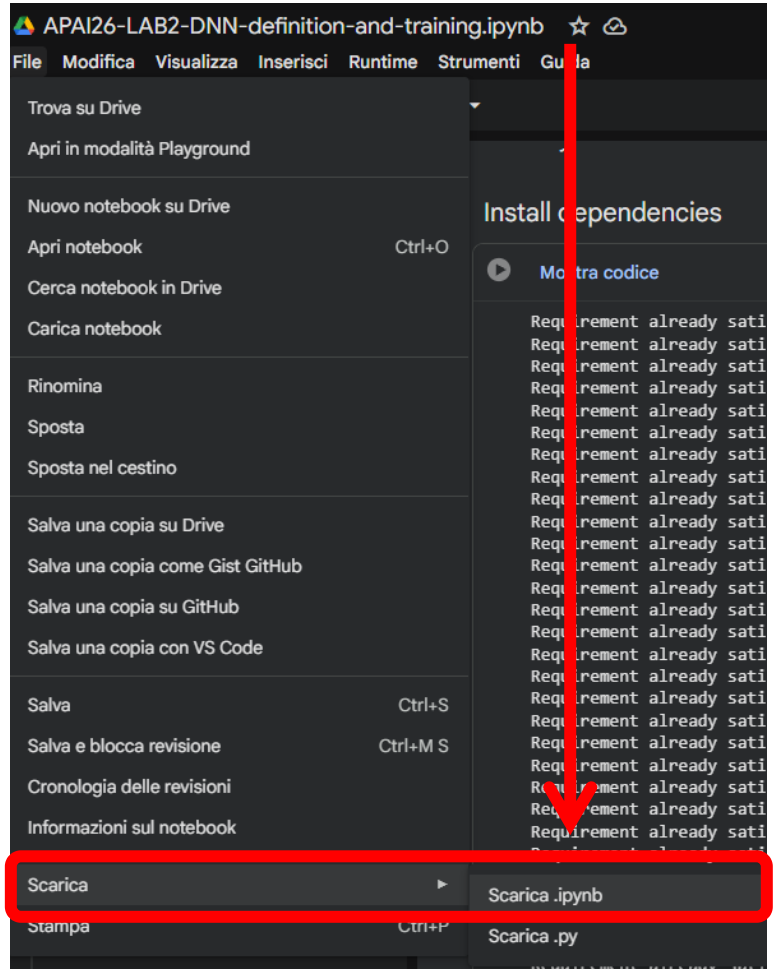
- The class is meant to be interactive: coding together and on your own!

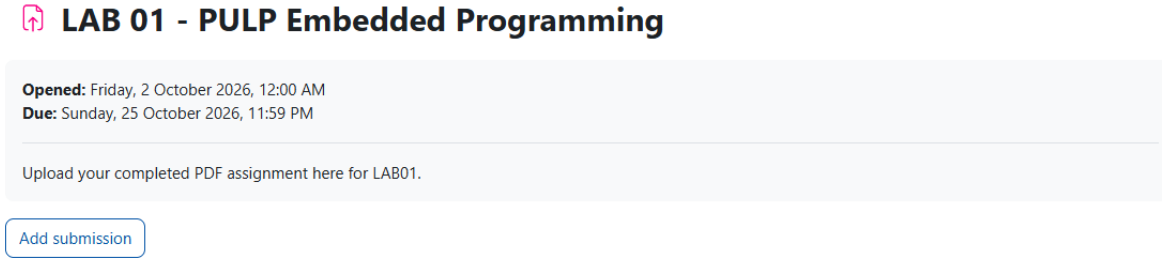
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

01. Lab Information – How to Deliver the Assignment

- You will solve the assignment **inside the given Python notebook** and deliver **the notebook** on Virtuale
  - **Run the cells and save the notebook**, so the results show up in the cells
  - **Rename the file** to *LAB<number_of_the_lesson>_APAI_<name_surname>.ipynb*
  - **If you run locally,** upload the saved notebook directly from your PC
  - **If you run on Colab**: download it first (*File → Download → Download .ipynb*), then upload it
  - Use **Virtuale to upload the notebook** before the **deadline (Sunday 1st of November, end of day)**
  - For each lab that you solve and submit successfully, you will receive **1 extra point**, up to a **maximum of 4 points**





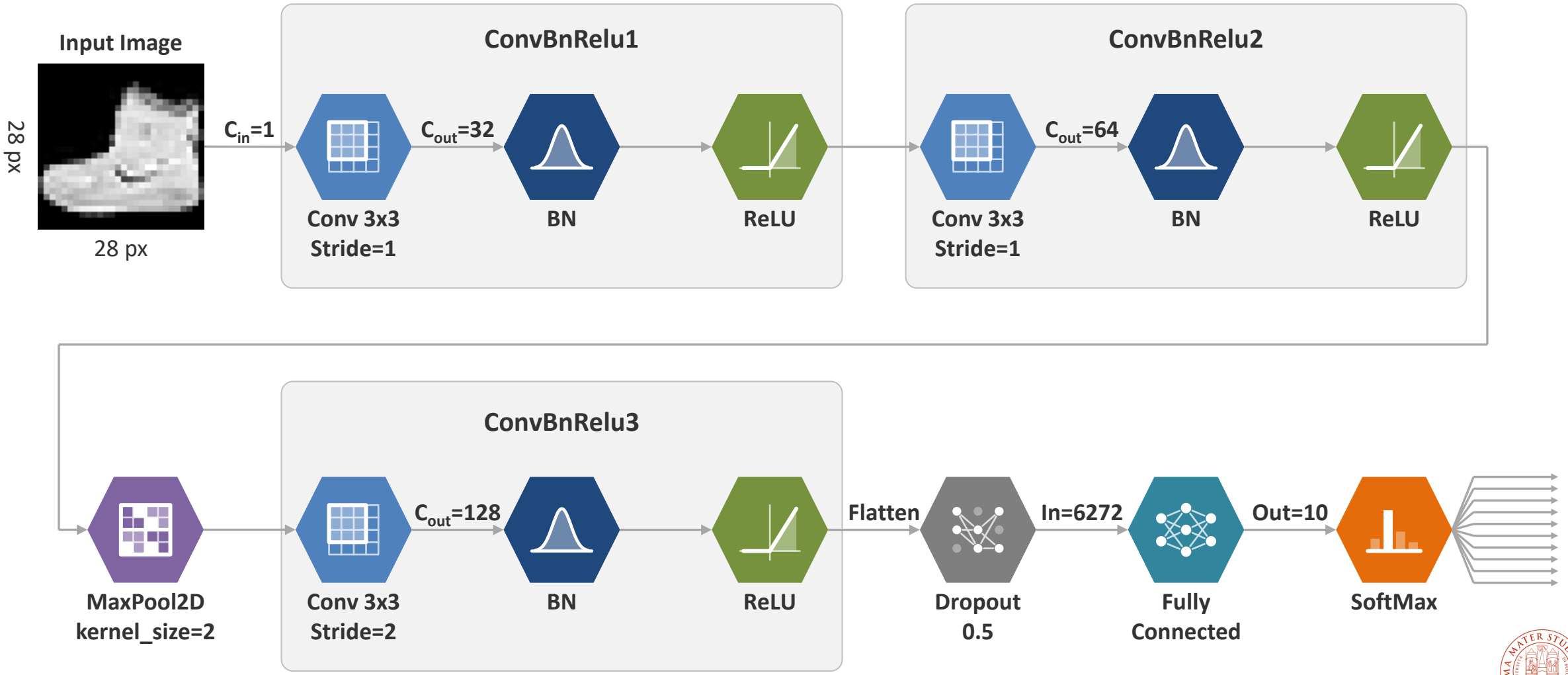


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



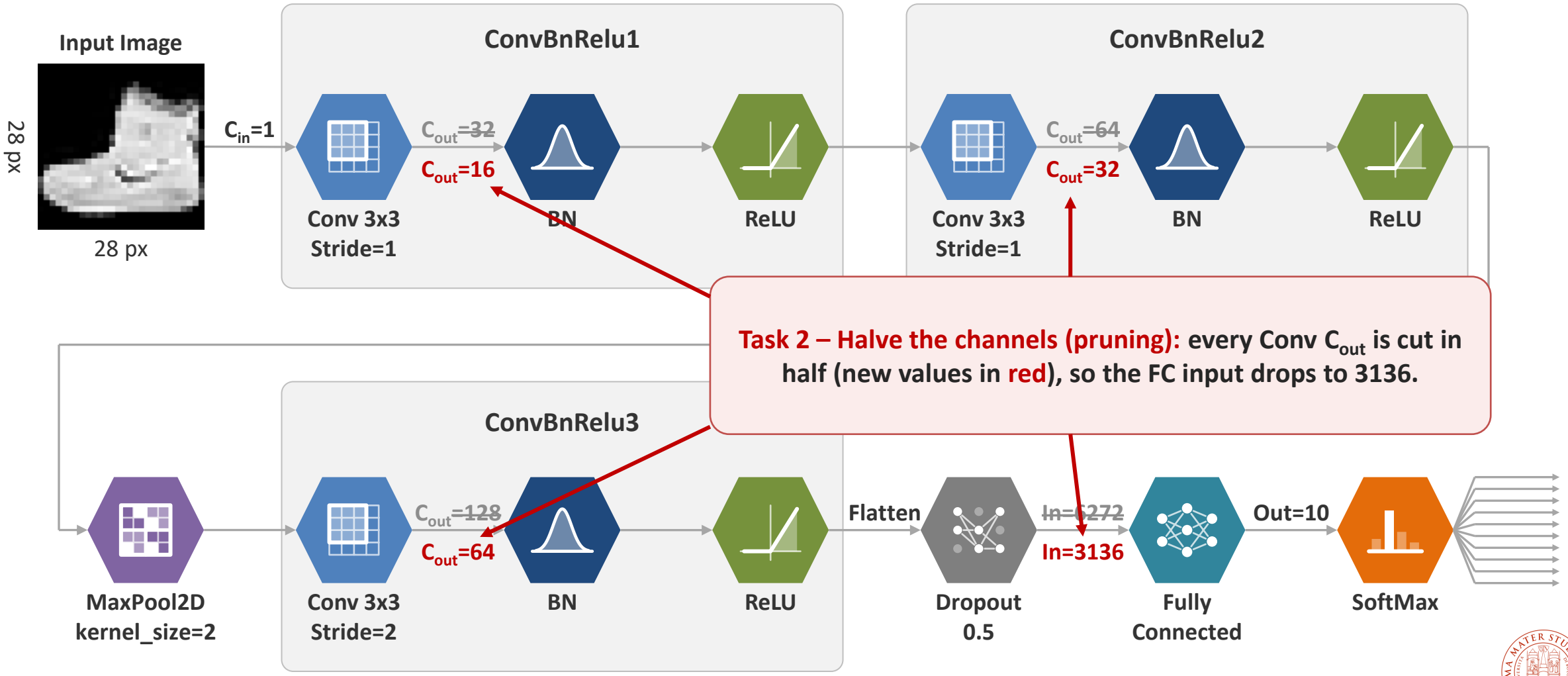
02. Introductory Information

02. Intro Information – Same CNN as lab 02



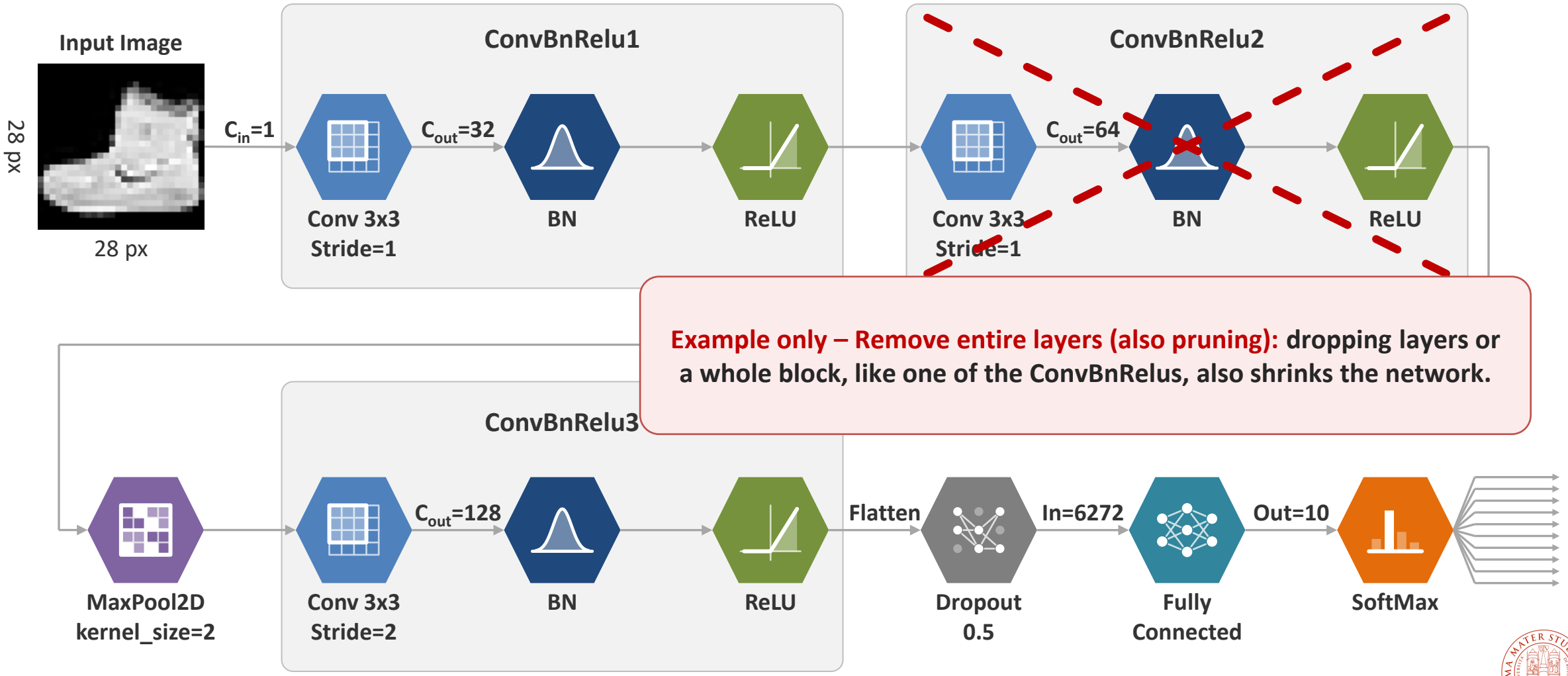
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – DNN Shrinking Techniques



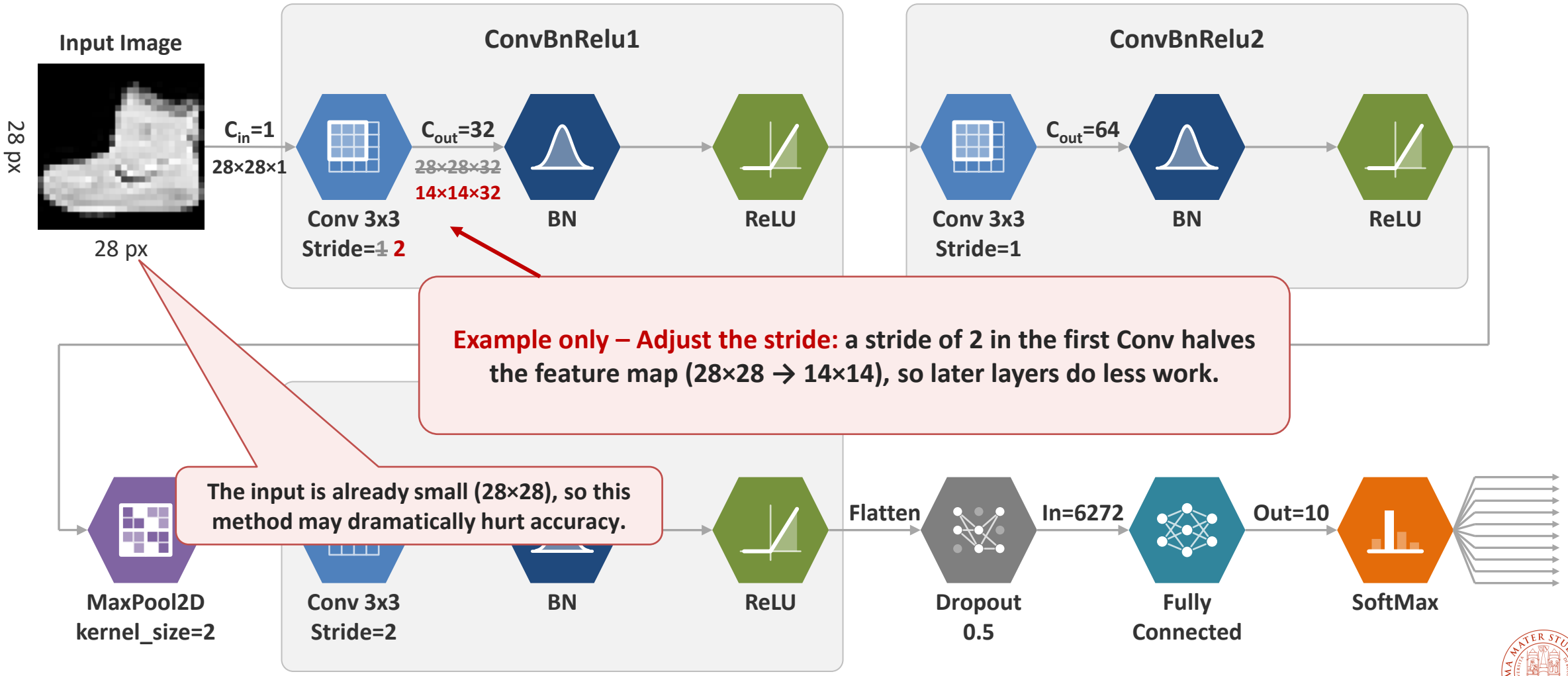
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – DNN Shrinking Techniques



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – DNN Shrinking Techniques



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – DNN Shrinking Techniques

Halve the channels

Remove layers

Adjust the stride

Retraining is needed in all cases

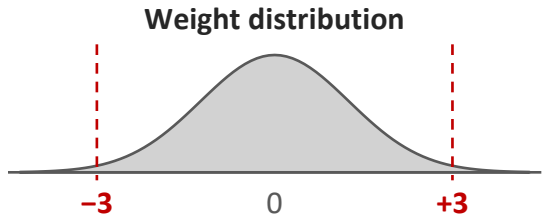
After any shrinking step, the smaller network must be trained again to recover accuracy (the weights of a layer are trained for a specific distribution and positioning of input activations).

Fewer epochs from a pretrained model

- Starting from the pretrained weights, the shrunk network needs fewer epochs than training from scratch.
- Their remaining layers already embed the features learned during the original training.

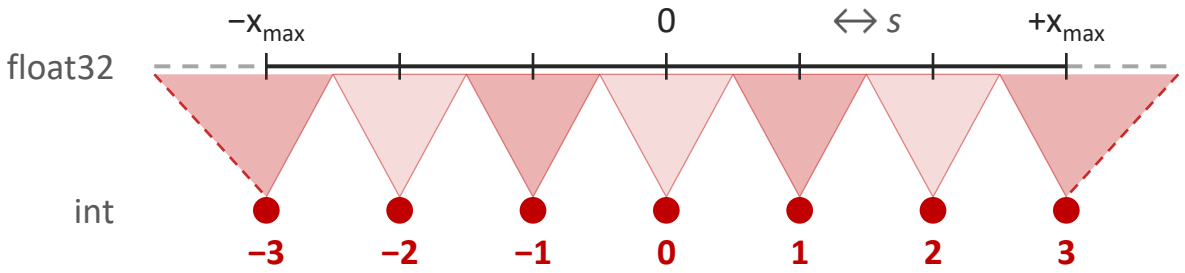
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – Post-Training Quantization

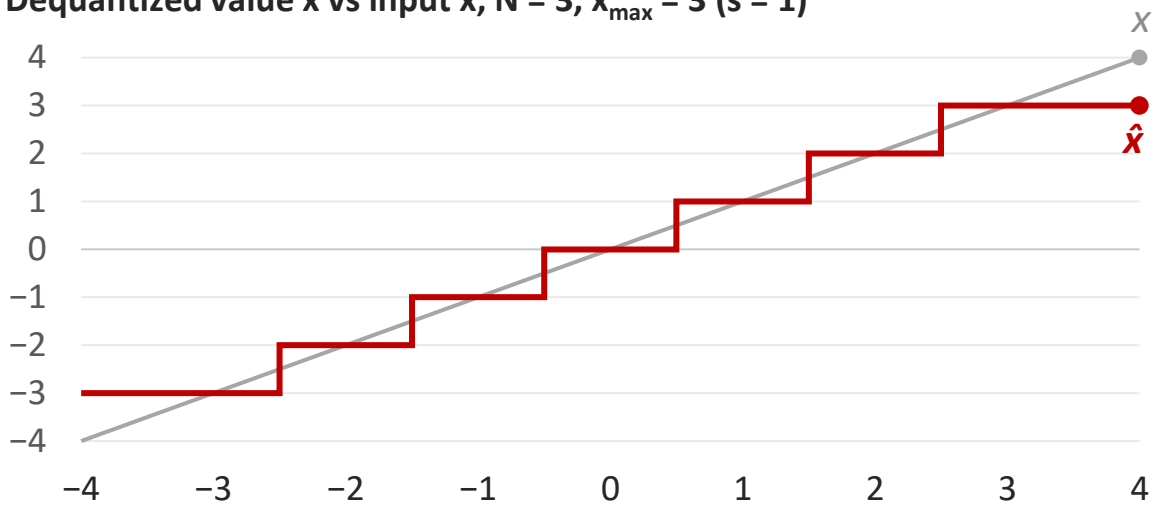


- **“Light” technique:** the trained network is quantized **without retraining**.
- **float32 → integer:** a smaller model (8-bit values take 4× less memory than float32) and potentially faster inference, since integer operations are cheaper than float ones.

From float32 to integers (N = 3, so q_{max} = 3)



Dequantized value x̂ vs input x, N = 3, x_{max} = 3 (s = 1)



Symmetric, per-tensor, linear quantization to N bits

$$q = clamp(\ round(\ x / s\ ),\ -q_{max},\ q_{max}\ )$$

$$q_{max} = 2^{N-1} - 1 \qquad s = x_{max} / q_{max}$$

$$\hat{x} = s \cdot q$$

s: scaling factor $\mathbf{x_{max}}$: clipping range $\mathbf{x - \hat{x}}$: quantization error

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – Brevitas & Fake Quantization

- **Brevitas** (open source, AMD Xilinx) is our quantization framework, built on PyTorch: xilinx.github.io/brevitas
- **Replace the nodes:** each layer becomes a quantization-aware one whose `forward` quantizes the weights and, optionally, the activations.
- **Better ranges from statistics:** these nodes record weight and activation distributions, stored as float **scaling factors** and **zero-offsets**.
- **Fake quantization (FQ):** linear quantizers sit in the graph, but operands stay float, so the model is still fully trainable.
- **Layer-wise**: we quantize one **`nn.Module`** at a time (a `torch.fx` trace is the alternative); Brevitas automates most of the work.

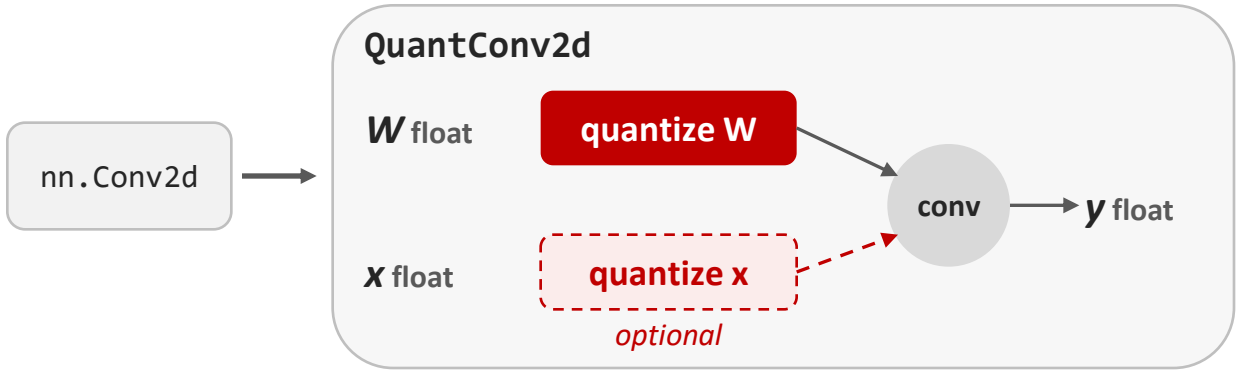
What fake quantization does to the weights (N = 3)



float32: continuous values

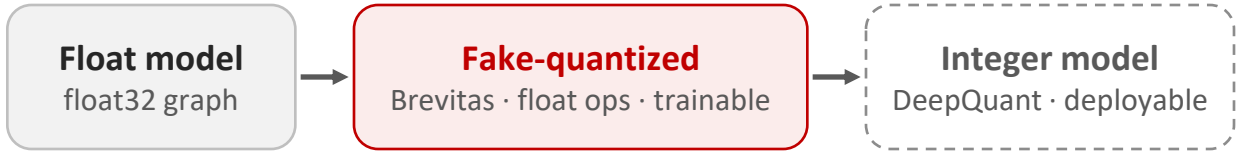
7 levels k·s, still stored as float

Step 1: a float layer becomes a quantization-aware layer



Quantizers use s and z set from collected statistics; the conv still runs in float.

From float model to integer model

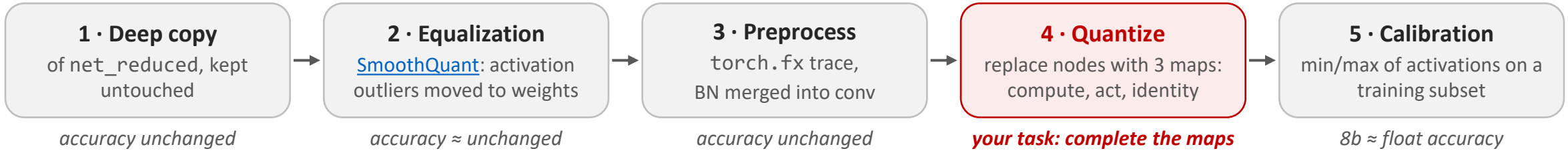


FQ is not deployable: scales are applied in float, the 32b → 8b requantization is missing, and it is not bit-true to integer hardware. DeepQuant converts it to a full-integer graph.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – FQ Step 1: Float-to-Fake

Float-to-fake workflow



Forward uses rounded values, backward skips the rounding



- **Why keep float32:** gradient updates are tiny; stored as integers, most would be rounded away.
- **Why STE:** the gradient of round is zero almost everywhere, so it is treated as the identity.
- **We use PTQ** (calibration only). **QAT** would add a few fine-tuning epochs on the FQ model.

Our quantizer choices

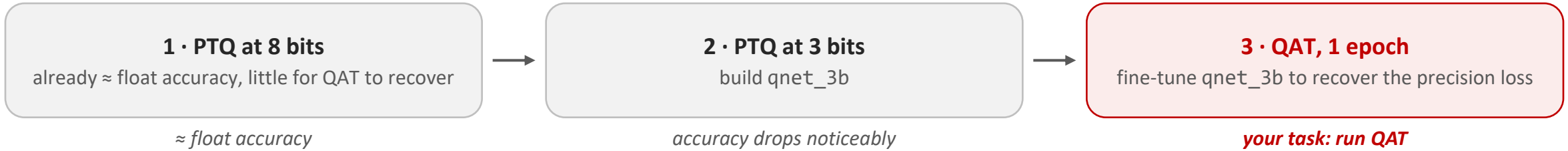
`Int8` signed 8b · `Weight` for weights · `PerTensor` one scale per layer · `Float` float scale — *= Brevitas name*

Tensor	Setting
Weights	8b signed, symmetric, one scale per layer (wmin…wmax)
Inputs / outputs	8b signed; range from calibration (99.999th percentile)
Biases	32b, added to the 32b MAC accumulator
Scaling factors	float; power-of-two would mean shifts, but less accuracy

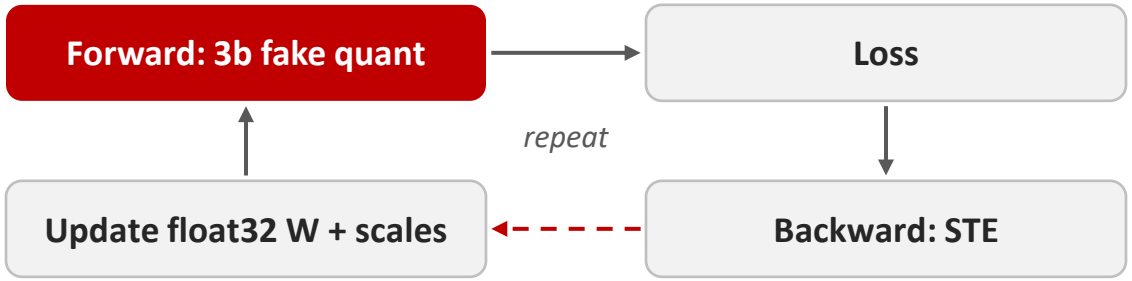
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – FQ Step 2: Quantization-Aware Training

Quantization-aware training workflow



One QAT epoch: training through the fake-quantized network

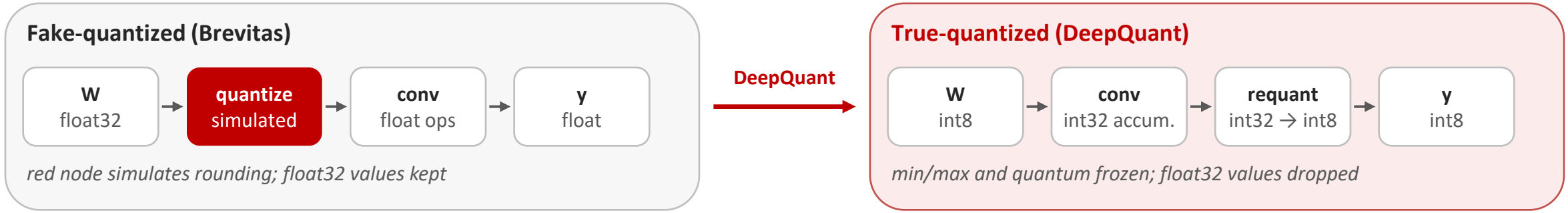


- **QAT** pays off at very low bit widths: at 8b PTQ already matches float.
- **No re-calibration:** activation scales are trained with the weights, and calibrating again would overwrite them.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – FQ Step 3: Fake-to-True

One layer before and after true quantization



- **True Quantization (TQ):** the fake-quantization nodes are removed, and every parameter becomes an integer.
- **Freeze the ranges:** min, max and the input scaling factor (*quantum*) become constants.
- We deploy the 8-bit **`qnet`**, fake-quantized with PTQ in step 1.
- The **accumulation** in the conv computations is done in **int32 to avoid overflow**, and then **requantization with the same parameters** as before.
- **DeepQuant** (PULP platform) turns Brevitas FQ output into TQ automatically.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Intro Information – ONNX & Netron

ONNX · Open Neural Network Exchange

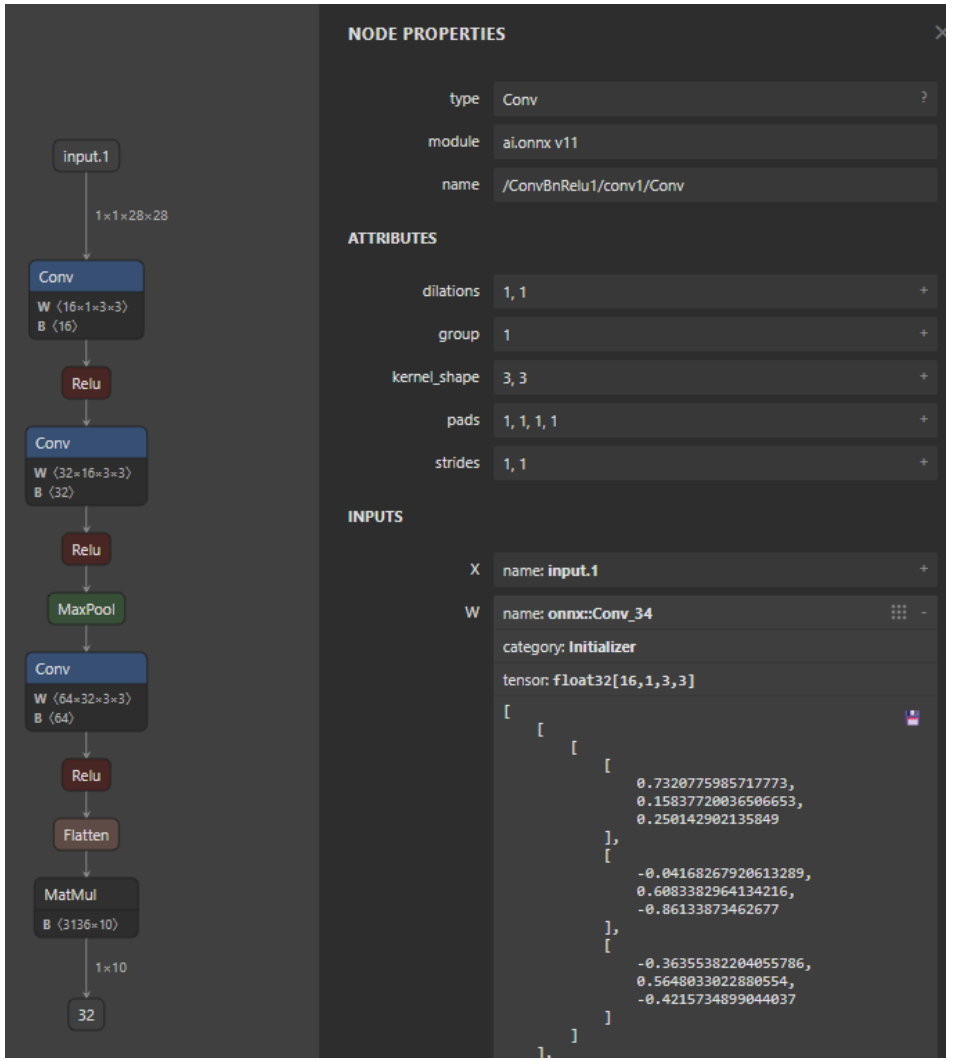
- An attempt to **standardize how neural networks are represented**: one open file format and a common set of operators, developed by Facebook and Microsoft.
- A model exported from PyTorch, TensorFlow and others can be read by any compatible tool, runtime or hardware compiler.

Docs: onnx.ai · GitHub: github.com/onnx/onnx

Netron · free visualizer for ONNX

- Open an **.onnx** file to see the graph, then click a node for its type, attributes, inputs and outputs (figure).
- Runs in the browser, as a desktop app, or inside **VS Code** (search “Netron” in Extensions).

Web app: netron.app · GitHub: github.com/lutzroeder/netron



Our shrunk network in Netron: graph and first Conv's properties

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



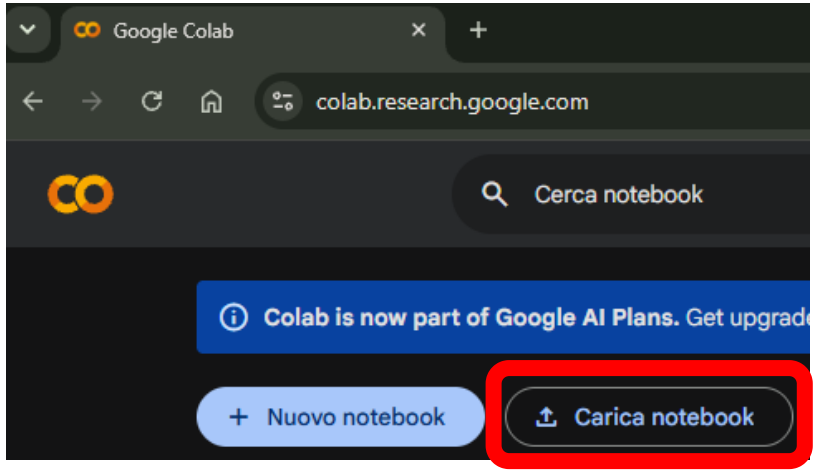
03. Setup

03. Setup – Google Colab

- **THIS LAB MUST BE RUN ON YOUR OWN PERSONAL COMPUTER**, since the lab machines have very limited internet access, which wouldn't allow you to access either Google Colab, or to download the necessary libraries
- There are 2 paths of completing the assignment
  - Running locally, inside VS Code – open the notebook in VS Code and it should work with minimal setup; VS Code will direct you to install the missing components if that's the case
  - Running on Google Colab – an online tool for running Python Notebooks, on cloud, accessing it via your browser (so it should be universally compatible), following the steps below

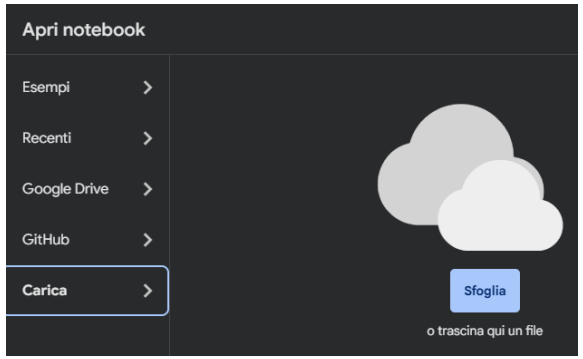
Colab steps:

- Step 1: Download the Notebook, either from Virtuale, or clone from the repo
- Step 2: Access Google Colab and choose the "Upload Notebook"/"Carica notebook" option

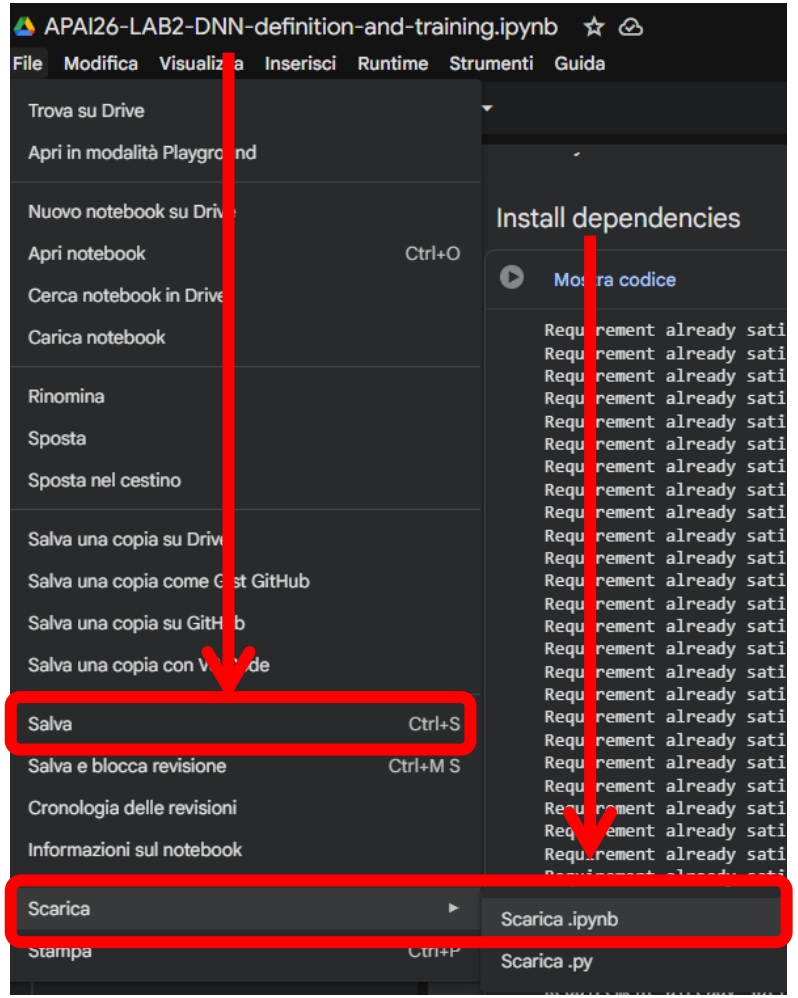


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Setup – Google Colab



- Step 3: Use the pop-up to upload the notebook that you just downloaded
- Step 4: Go through the tasks and requirements, and solve the assignments
  - If you have questions at any point, feel free to ask!
- Step 5: Once you are happy with the results and have completed all tasks, be sure you do NOT delete the content of any of the run cells and that everything is properly saved.
- Step 6: Now, follow the Colab download instructions from the first part and upload the completed Notebook to Virtuale



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Călin Diaconu – calin.diaconu2@unibo.it

Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it

Ihor Sachok – ihor.sachok@unibo.it

Marco La Barbera – marco.labarbera2@unibo.it